PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering (UE22CS351A) | Lab 6: Code and Branch Coverage Analysis

Name: Samrudh Patil | SRN: PES1UG24CS417

1. Deliverable Links & Repository Details

Item	Link / Details
Assigned Starter Repository	https://github.com/SETAPESU26/L6_L7_52_visa_application
Individual GitHub Repository	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417
Lab Submission Folder	https://github.com/Samrudh7-0045/SE-Labs-PES1UG24CS417/tree/main/LAB-6
Google Form Submission Link	https://forms.gle/7KzWpGWLqh35M9dj8
Coverage Achieved	**100% Statement Coverage (37/37) | 100% Branch Coverage (26/26)**

2. Overview: Code Coverage vs. Branch Coverage

In this lab, we used Python's coverage library and pytest to test the visa_application.py module. The goal was to understand how code coverage and branch coverage differ in practice and ensure both reach over 90% (achieved 100% for both):

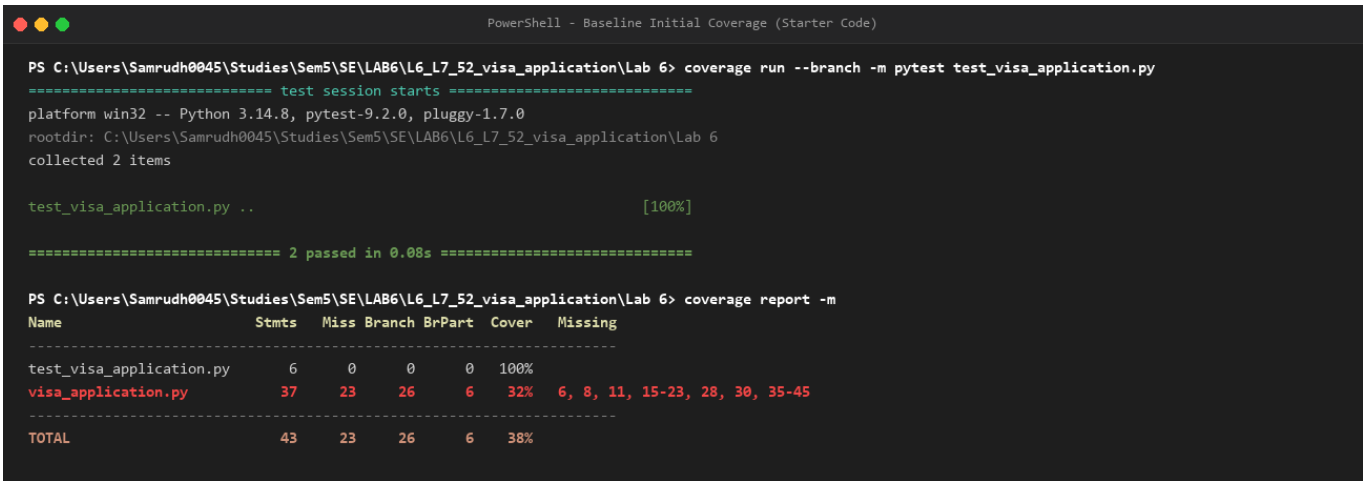
- **Line Coverage (Code Coverage):** Measures if every line of code executed at least once during testing. It helps find unreached lines or dead code, but it does not check if conditional statements evaluated to both True and False.
- **Branch Coverage:** Checks whether every branch of every control statement (like if, elif, else, and boolean compound conditions) was taken. This makes sure both the True and False outcomes are tested.
- **Why Line Coverage Alone Isn't Enough:** If an if statement has no else block, testing it only when the condition is True will execute all lines inside it and show 100% line coverage. However, the case where the condition is False is completely skipped and untested. Branch coverage catches this gap.

3. Baseline Initial Coverage (Starter Code)

The assigned starter repository contained two basic tests in test_visa_application.py. Running the initial coverage showed that most functions and logical branches were not tested yet:

Metric	Initial Result	Untested Areas
Statement Coverage	**38%** (14 / 37 statements)	Missing lines: 6, 8, 11, 15-23, 28, 30, 35-45
Branch Coverage	**32%** (6 / 26 branches)	20 branches untested across all three functions

Terminal Output: Baseline Coverage Run (38% Line, 32% Branch)

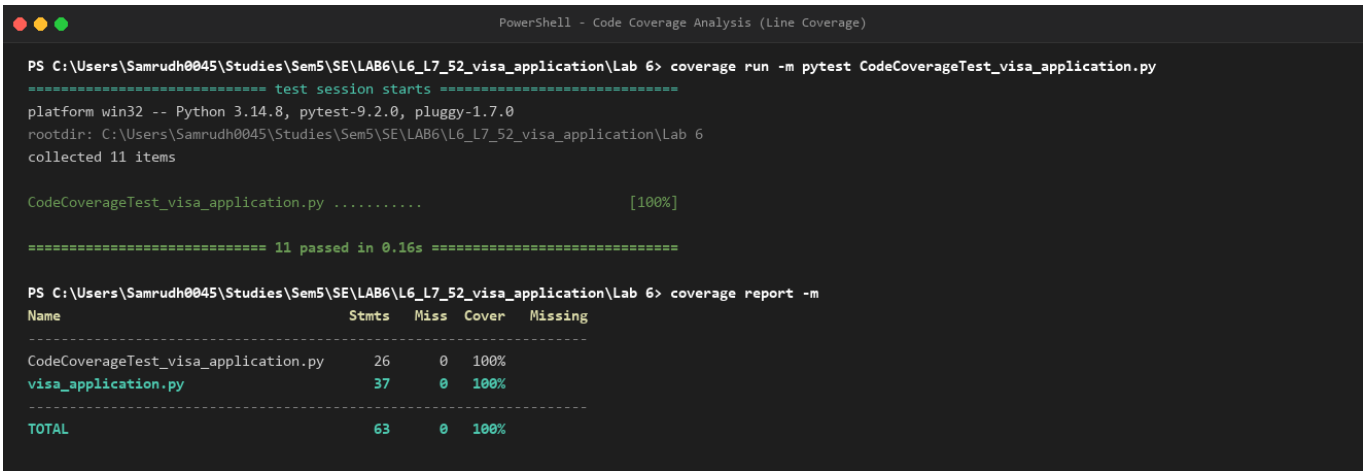


4. Code Coverage Testing (100% Statement Coverage)

To meet the requirement of achieving at least 90% code coverage, I created a dedicated test file **CodeCoverageTest_visa_application.py** with 11 targeted test cases to execute every statement in visa_application.py:

- **visa_fee():** Tested invalid visa type error, negative entries error, nationality group 'A' extra entry increment, zero extra entries return, express speed rate, normal speed rate, and surcharge calculations for entries over 5.
- **documents_complete():** Tested renewal path returning True, non-renewal path with low photos returning False, and normal submission with valid forms returning True.
- **application_status():** Tested express peak open status, normal off-peak closing status, and normal off-peak closed status.

Terminal Output: Code Coverage Analysis (100% Statement Coverage)



Test Cases in CodeCoverageTest_visa_application.py:

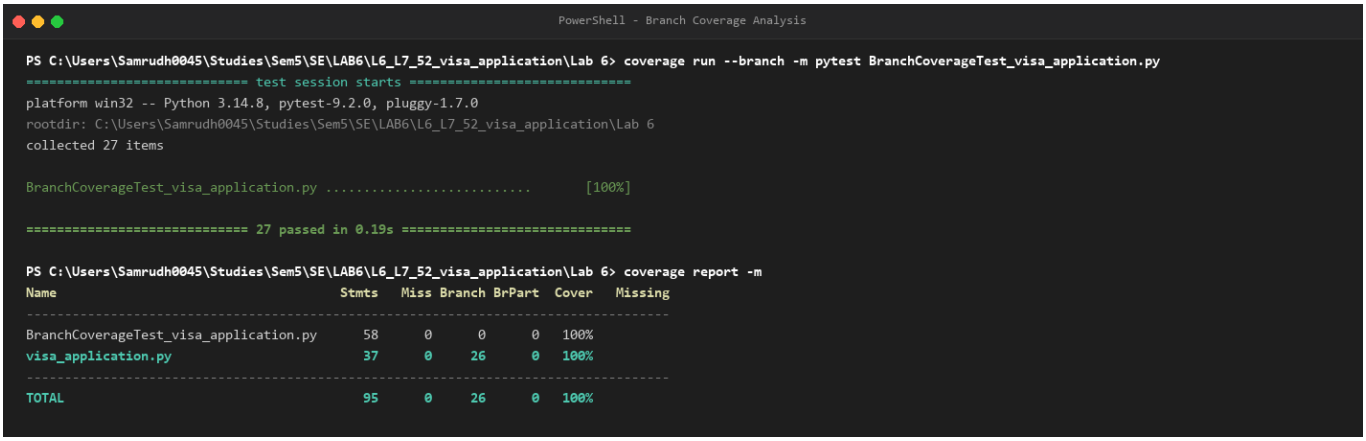
Test Case	Function	Logic Covered	Expected Output
test_visa_fee_unknown_type	visa_fee	Invalid visa type check (line 5-6)	ValueError('unknown visa type')
test_visa_fee_invalid_entries	visa_fee	Negative entry count check (line 7-8)	ValueError('invalid entries')
test_visa_fee_included_nationality_a	visa_fee	Nationality group A bonus entry (line 10-14)	Fee = 0
test_visa_fee_express_high_extra	visa_fee	Express base 100 + volume 20 + express 10	Fee = 910
test_visa_fee_normal_speed_extra	visa_fee	Normal speed rate 40 with extra <= 5	Fee = 40
test_documents_complete_renewal	documents_complete	Renewal application with forms >= 1	True
test_documents_complete_low_photos	documents_complete	Non-renewal with photos < 2	False
test_documents_complete_standard	documents_complete	Standard non-renewal with photos >= 2, forms >= 3	True
test_app_status_express_peak_open	application_status	Express + peak (cutoff 25), days left 25	'open'
test_app_status_standard_closing	application_status	Normal off-peak (cutoff 30), days left 20	'closing'
test_app_status_standard_closed	application_status	Normal off-peak (cutoff 30), days left 10	'closed'

5. Branch Coverage Testing (100% Branch Coverage)

Even though statement coverage reached 100%, branch coverage revealed missing paths. For example, in visa_fee(), when extra > 5 was tested only with express speed, the branch where speed is normal (line 21 → 23) was skipped. Also, in compound condition if nationality_group == 'A' and visa_type != 'student':, testing when nationality is 'A' but visa type is 'student' is required to test the second operand evaluating to False.

To get 100% branch coverage, I created **BranchCoverageTest_visa_application.py** with 27 test cases covering all 26 decision branches:

Terminal Output: Branch Coverage Analysis (100% Branch Coverage, 26/26 Branches)



Branch Coverage Breakdown (All 26 Branches Covered):

Branch ID & Line	Condition / Decision Point	Paths Tested	Test Input Details
B1, B2 (Line 5)	visa_type not in INCLUDED_ENTRIES	True / False	Unknown type 'work' raises error; valid types proceed
B3, B4 (Line 7)	entries < 0	True / False	Negative entries (-1) raises error; entries >= 0 proceed
B5, B6, B7 (Line 10)	nationality_group == 'A' and visa_type != 'student'	True / False (Left) / False (Right)	1. Group A + tourist (both True) 2. Group B + tourist (left False) 3. Group A + student (right False)
B8, B9 (Line 13)	extra <= 0	True / False	Extra <= 0 returns 0; extra > 0 proceeds to rate calculation
B10, B11 (Line 15)	speed == 'express' ... else	True / False	Express sets rate = 100; normal sets rate = 40
B12, B13 (Line 19)	extra > 5	True / False	Extra = 6 adds 20; extra <= 5 keeps base rate
B14, B15 (Line 21)	speed == 'express' (inside extra > 5)	True / False	Express adds extra 10; normal keeps rate + 20 (covers 21->23)
B16, B17 (Line 27)	is_renewal	True / False	Takes renewal branch vs standard application branch
B18, B19 (Line 28)	forms_filed >= 1 (renewal)	True / False	Forms = 1 returns True; forms = 0 returns False
B20, B21 (Line 29)	photos < 2 (non-renewal)	True / False	Photos = 1 returns False; photos = 2 checks forms count
B22, B23 (Line 31)	forms_filed >= 3 (non-renewal)	True / False	Forms = 3 returns True; forms = 2 returns False
B24, B25 (Line 35)	is_express ... else	True / False	Express sets cutoff = 10; normal sets cutoff = 30
B26 (Line 39, 41, 43)	Peak season offset (+15) and status ranges	Open / Closing / Closed	Tested open, closing, and closed across all 4 cutoff combinations (10, 25, 30, 45)

6. Comparison & Git Commits

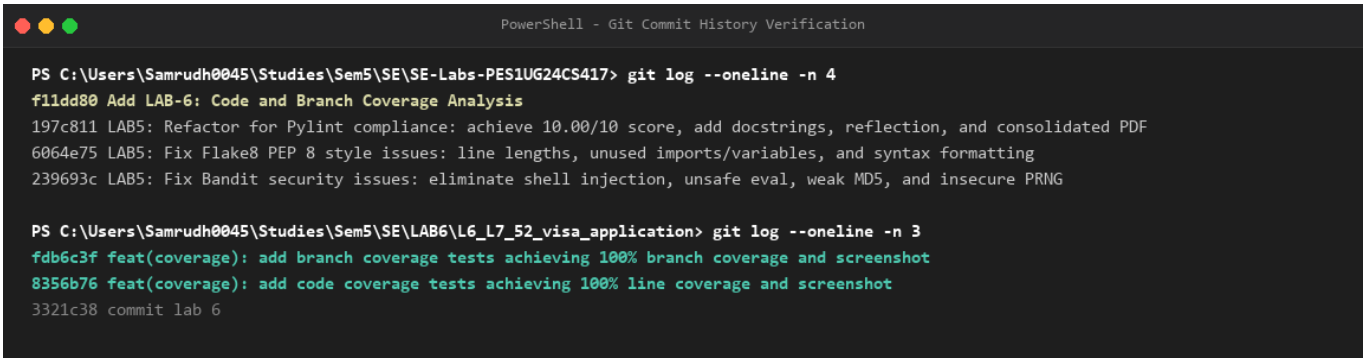
Summary comparing the initial starter code against the code coverage and branch coverage test suites:

Metric	Starter Suite	Code Coverage Suite	Branch Coverage Suite
Test File	test_visa_application.py	CodeCoverageTest_visa_application.py	BranchCoverageTest_visa_application.py
Number of Tests	2	11	**27**
Statements Tested	14 / 37 (38%)	**37 / 37 (100%)**	**37 / 37 (100%)**
Branches Tested	6 / 26 (32%)	25 / 26 (98%)	**26 / 26 (100%)**
Missing Lines / Branches	23 lines, 20 branches	0 lines, 1 branch (21->23)	**0 missing lines, 0 missing branches**

Git Commit History:

As instructed, separate commits were maintained for code coverage and branch coverage:

Terminal Output: Git Commits for Code and Branch Coverage



7. Reflection Questions & Analysis

1. What is the difference between Line Coverage and Branch Coverage? Give an example from visa_application.py.

Answer: Line coverage only checks whether the interpreter executed a given line of code during testing. Branch coverage checks whether every decision condition evaluated to both True and False.
In visa_application.py, lines 19–22 show this clearly:

```
if extra > 5:
    rate = rate + 20
    if speed == 'express':
        rate += 10
```

If we only test with extra = 7 and speed = 'express', every single line runs, giving 100% line coverage. But we never tested what happens when extra > 5 and speed = 'normal'. In that case, the inner condition is False and the extra 10 fee is skipped. Branch coverage caught this missing path (branch 21->23).

2. Why can 100% line coverage give a false sense of security?

Answer: 100% line coverage only shows that no line was completely skipped. It does not prove that the code handles all alternate scenarios. For example, if an if statement has no else, testing only the True condition executes all lines inside it while completely ignoring the False case. If the code relies on a variable keeping its default value or falls through to unexpected logic when the condition is False, bugs can easily slip into production unnoticed.

3. How should compound conditions like nationality_group == 'A' and visa_type != 'student' be tested?

Answer: In Python, the and operator uses short-circuit evaluation. To properly test a compound condition, you need test cases for each combination of outcomes:
• **Both True:** nationality_group = 'A' and visa_type = 'tourist' → whole condition is True, extra entry added.
• **Left False:** nationality_group = 'B' and visa_type = 'tourist' → short-circuits to False.
• **Left True, Right False:** nationality_group = 'A' and visa_type = 'student' → evaluates both operands and branches to False.
Testing these three scenarios ensures every logical path of the condition is validated.

4. What is the trade-off between reaching 100% branch coverage and writing meaningful assertions?

Answer: High coverage numbers alone don't guarantee that the tests are actually checking for correct behavior. A test can execute every branch without asserting anything, which gives 100% coverage on paper but tests zero functionality. While 100% branch coverage is great for finding untested paths, tests must always have strict assertions checking the exact return values (like checking if the fee equals 910 or 780, or checking specific exception types). Coverage measures how much code was run, while assertions verify that the code actually works correctly.

5. How does adding coverage checks to CI/CD pipelines improve code quality?

Answer: When a minimum coverage threshold (like 90%) is enforced in CI/CD pipelines, any pull request that adds new features or bug fixes without writing corresponding unit tests will fail the build automatically. This prevents untested code from reaching the main branch, catches regressions early, and makes sure testing standards stay consistent across the entire team.